

RISC Principles and ARM64 Architecture

RISC Design Philosophy, Modern Load/Store Model, AArch64 Programmer-Visible State, and Core Instruction Classes

Computer Organization & Architecture | Topic 2

Topic 2 Outline

Topic Objective

Analysis of the architectural philosophy of Reduced Instruction Set Computers (RISC) and examination of how modern ARM64 / AArch64 realizes this model via orthogonal 64-bit registers, condition flags, and instruction classes.

- **Section 1:** RISC Design Principles and the Load/Store Paradigm
- **Section 2:** CISC versus RISC Architectural Trade-Offs
- **Section 3:** AArch64 Architectural Overview and Encoding Model
- **Section 4:** Registers and 64-Bit Programmer-Visible State (x0–x30, xzr, sp, pc, pstate)
- **Section 5:** Core AArch64 Instruction Classes: Data Processing, Memory, and Control Flow
- **Section 6:** Summary and Worked Self-Test Exercises

Section 01

RISC Design Principles & the Load/Store Paradigm

Foundational RISC Design Principles

Reduced Instruction Set Computers (RISC) optimize the hardware-software interface by prioritizing simplicity and regularity:

- **Simpler Control Circuitry:** Regular instruction formats and orthogonal opcodes drastically reduce decode and control unit complexity.
- **Smaller Silicon Footprint:** Eliminating complex microcoded multi-cycle instructions frees die area for larger register files and caches.
- **Lower Energy Consumption:** Streamlined decoding reduces gate switching activity, yielding high performance-per-watt.
- **Predictable Pipelining:** Uniform fixed-width instruction lengths ensure smooth, hazard-free pipeline flow.

The Load/Store Architectural Paradigm

Separation of Memory Access and Computation

In a pure RISC architecture, data processing instructions (arithmetic, logic) operate **exclusively on registers**. Memory is accessed strictly through dedicated **load** and **store** instructions.

Arithmetic Isolation

- ALU inputs must reside in the register file.
- ALU outputs are committed to a register.
- No arithmetic instruction can target a memory address directly.

Explicit Memory Staging

- `ldr`: Reads data from memory into a register.
- `str`: Writes data from a register into memory.
- Simplifies pipeline address generation and memory stages.

Executing Memory Mutation in AArch64

To mutate a 64-bit variable in RAM (e.g., $x = x + 5$), AArch64 strictly sequences three distinct operations:

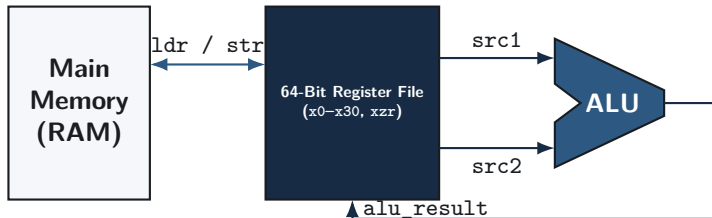
- ❶ **Load Stage:** Read 64-bit value from memory base pointer `[x0]` into register `x1`.
- ❷ **Compute Stage:** Perform 64-bit register addition using immediate value `#5`.
- ❸ **Store Stage:** Write modified 64-bit value in `x1` back to memory address `[x0]`.

```
ldr x1, [x0]           // 1. Load 64-bit x from RAM into register x1
```

```
add x1, x1, #5         // 2. Perform arithmetic within register file
```

```
str x1, [x0]           // 3. Write updated 64-bit result back to RAM
```

Visualizing the Load/Store Isolation



- The ALU is completely decoupled from the memory bus.
- Memory interactions occur exclusively across the 64-bit register file boundary.

CISC versus RISC

Architectural Trade-Offs

Complex Instruction Set Computers (CISC)

CISC architectures maximize work accomplished per individual instruction:

- **Rich, Multi-Step Operations:** A single instruction combines memory fetching, arithmetic calculation, and memory write-back.
- **Direct Memory Arithmetic:** Instructions can operate directly on operands residing in memory (e.g., `add [rax], rbx`).
- **Variable-Length Formats:** Instruction encodings range from 1 to 15 bytes, requiring complex multi-stage decoders.
- **Dense Static Code:** High semantic density minimizes program size in RAM.

Design Penalty

Heavy silicon area and energy cost allocated to instruction decoding and complex hardware microcode control paths.

Code Comparison: Implementing $x += 5$

Consider incrementing a 64-bit memory variable x by immediate constant 5:

CISC Implementation (x86-64)

- **Instruction:** `add QWORD PTR [x], 5`
- Single variable-length instruction.
- CPU decodes operand, issues memory read, performs ALU add, and writes to memory in microcode.
- Compact binary size, but complex execution timing.

RISC Implementation (ARM64)

- **Instruction Stream:**
 - `ldr x1, [x0]`
 - `add x1, x1, #5`
 - `str x1, [x0]`
- Three fixed 32-bit instructions.
- Higher instruction count, but simple 1-cycle decode and pipeline predictability.

CISC vs. RISC Trade-Off Matrix

Dimension	CISC (e.g., x86-64)	RISC (e.g., AArch64, RISC-V)
Instruction Size	Variable (1 to 15 bytes)	Fixed (strictly 32-bit)
Memory Access	Combined with computation	Strict Load/Store only
Addressing Modes	Many, complex modes	Few, regular base-register modes
Decode Complexity	High (requires decode ROM/ μ ops)	Low (hardwired, orthogonal fields)
Register Count	16 visible GPRs (rax-r15)	31 visible GPRs (x0-x30)
Pipelining	Difficult due to multi-cycle instructions	Highly efficient and deterministic

AArch64 Architectural Overview and Encoding Model

ARM64 (AArch64) Architecture Overview

AArch64 is ARM's modern 64-bit RISC instruction set architecture designed for clean, scalable, high-performance computing:

- **Modern Lineage:** Introduced with ARMv8-A; powers Apple Silicon (M-series), AWS Graviton, and flagship mobile processors.
- **Cleaned Architecture:** Removed legacy ARM32 complexities (such as conditional execution fields on every opcode and exposed pipeline prefetch offsets).
- **Orthogonal Register Set:** 31 general-purpose 64-bit registers (x0–x30) plus dedicated sp, pc, and zero register xzr.

Architectural Contract vs. Microarchitecture

The AArch64 ISA guarantees identical software behavior across energy-efficient IoT cores and high-throughput server chips.

AArch64 Fixed-Width 32-Bit Instruction Encoding

In AArch64, every instruction is encoded into an exact **fixed 32-bit word**:



- **5-Bit Register Specifiers:** Exactly 5 bits each for rd, rn, and rm to index all 31 registers ($2^5 = 32$).
- **Size Flag (sf):** Bit 31 selects between 32-bit operation (w-registers, sf = 0) and 64-bit operation (x-registers, sf = 1).
- **Consistent Alignment:** Instructions are strictly 4-byte aligned in memory.

Dual Register Views: X (64-Bit) and W (32-Bit)

Each physical 64-bit register can be accessed in two distinct architectural widths:



Zero-Extension Rule

When an instruction writes to a 32-bit destination register (W_n), hardware automatically **zeroes out the upper 32 bits** (bits [63:32]) of the corresponding X_n register, preventing stale data leakage.

Registers and Programmer-Visible State

AArch64 Programmer-Visible Register File

AArch64 exposes **31 general-purpose registers** (x0–x30), plus special registers:

Register	Architectural Role	Standard Software Convention
x0–x7	Parameter / Return	Function arguments 0–7; x0–x1 return values
x8	Indirect Result	Location pointer for large struct returns
x9–x15	Caller-Saved	Temporary scratch registers
x16–x17	IP0 / IP1	Intra-procedure call scratch registers
x18	Platform Register	Reserved for platform/OS runtime use
x19–x28	Callee-Saved	Preserved across function calls
x29 (fp)	Frame Pointer	Points to base of current stack activation record
x30 (lr)	Link Register	Holds return address for function calls (b1)

Special Architectural Registers in AArch64

AArch64 eliminates ARM32 register aliasing by introducing dedicated registers:

- **Zero Register (xzr / wzr):**
 - Reads always return numerical value 0; writes are silently discarded.
 - Encoded as register specifier 31 in most data-processing opcodes.
- **Stack Pointer (sp / wsp):**
 - Dedicated register for stack alignment (strictly 16-byte aligned at runtime).
 - Encoded as register specifier 31 in load/store and stack arithmetic.
- **Program Counter (pc):**
 - Holds current instruction address; **not a general-purpose register**.
 - Cannot be read or modified by arithmetic instructions (no `add pc, pc, #4`).
 - Updated exclusively by branch, call (b1), and exception return instructions.

Processor State (PSTATE) and Condition Flags

In AArch64, status flags reside in specialized fields within **PSTATE** (accessed via NZCV):

N [31]	Z [30]	C [29]	V [28]	Reserved / System Control Bits	EL [3:2]
-----------	-----------	-----------	-----------	--------------------------------	-------------

The Arithmetic Condition Flags (NZCV)

- **N (Negative)**: Set if result bit 63 (or 31) is 1 (negative signed value).
- **Z (Zero)**: Set if the result of the operation equals 0.
- **C (Carry)**: Set if an unsigned addition generated a carry-out (or unsigned sub did not borrow).
- **V (oVerflow)**: Set if signed operation exceeded 64-bit 2's complement bounds.

Complete AArch64 Architectural State

Formal Definition

Architectural State comprises all hardware registers and memory contents that determine the exact execution context of software.

Programmer-Visible State

- 31 General Registers: x0–x30.
- Zero Register: xzr / wzr.
- Dedicated Stack Pointer: sp.
- Program Counter: pc.
- Condition Flags: PSTATE.NZCV.
- 64-Bit Linear Address Space (RAM).

Microarchitectural Separation

- Physical rename registers (128+ entries).
- Branch target buffers (BTB).
- L1/L2/L3 cache hierarchies.
- Speculative execution buffers.
- *All internal and invisible to software.*

Core AArch64 Instruction Classes

AArch64 Instruction Categories

Every operation in the AArch64 ISA partitions into three primary functional classes:

1. Data Processing

- Arithmetic & logic.
- Register-only computation.
- Suffix 's' sets flags (adds).
- Examples: add, sub, mul, mov, cmp.

2. Memory Access

- Base register addressing.
- Explicit sizing (b, h, w, x).
- Pre/post-indexing modes.
- Examples: ldr, str, ldp, stp.

3. Control Flow

- Unconditional branches.
- Conditional branches.
- Subroutine calls (bl).
- Examples: b, b.eq, b.ne, b.lt, ret.

Class 1: Data Processing Instructions

Data processing instructions compute mathematical or logical operations within registers:

Instruction Syntax	Semantic Operation	Architectural Effect
<code>add x3, x1, x2</code>	64-Bit Addition	$x3 \leftarrow x1 + x2$
<code>sub x3, x3, x2</code>	64-Bit Subtraction	$x3 \leftarrow x3 - x2$
<code>mul x3, x1, x2</code>	64-Bit Multiplication	$x3 \leftarrow x1 \times x2$
<code>mov x3, x4</code>	Register Copy	$x3 \leftarrow x4$
<code>and x0, x1, x2</code>	Bitwise Logical AND	$x0 \leftarrow x1 \text{ AND } x2$
<code>orr x0, x1, x2</code>	Bitwise Logical OR	$x0 \leftarrow x1 \text{ OR } x2$

32-Bit Variant

`add w3, w1, w2` performs 32-bit addition and zero-extends the upper 32 bits of `x3`.

Class 2: Memory Access (Load/Store)

Memory access instructions transfer data between RAM and the 64-bit register file:

```
ldr x1, [x2]    // Load 64-bit doubleword at address in x2 into x1
str w5, [x3]    // Store 32-bit word from w5 into memory address in
~
ldr x1, [x2, #8] // Immediate offset: Load from address (x2 + 8)
ldp x0, x1, [sp]           // Load Pair: Read two 64-bit words
simultaneously
```

Bracket Notation [x2]

Brackets represent **memory dereferencing**. In AArch64, memory base pointers must reside in 64-bit registers (x0–x30 or sp).

Comparison Operations and the `cmp` Instruction

Conditional decision-making begins with explicit comparison instructions:

`cmp x0, x1` (Alias for `subs xzr, x0, x1`)

- Performs internal 64-bit subtraction: $\text{Temp} \leftarrow x0 - x1$.
- **Discards Result:** Writes output to the zero register (`xzr`).
- **Updates NZCV:** Sets condition flags in `PSTATE` based on the subtraction.

If $x0 == x1$

$x0 - x1 = 0 \rightarrow$ **Z flag set to 1.**

If $x0 < x1$

$x0 - x1 < 0 \rightarrow$ **N flag set to 1.**

Class 3: Control Flow and Branching

Branch instructions alter program flow by loading new target addresses into the pc:

Unconditional Branch

`b target_label`

- Forces execution to diverge.
- `b1 label1`: Branch with Link (writes return address into `x30/1r`).
- `ret`: Returns to address in `1r (x30)`.

Conditional Branches

`b.<cond> target_label`

- Evaluates NZCV flags in `PSTATE`.
- If condition holds true $\rightarrow$ updates pc.
- If false $\rightarrow$ falls through ($PC \leftarrow PC + 4$).

Mnemonic	Meaning	Tested Flag Condition
<code>b.eq</code>	Branch if Equal	$Z == 1$
<code>b.ne</code>	Branch if Not Equal	$Z == 0$
<code>b.lt</code>	Branch if Less Than (Signed)	$N \neq V$
<code>b.gt</code>	Branch if Greater Than (Signed)	$(Z == 0) \text{ and } (N == V)$

Assembly Decisions: The Compare-and-Branch Pattern

High-level if-else statements are implemented via a 2-step assembly pattern:

Step 1: Evaluate Condition

```
cmp x1, x2
```

- Subtracts x2 from x1 internally.
- Updates **NZCV** flags in PSTATE.

Step 2: Test Flags & Branch

```
b.eq equal_label
```

- Evaluates flag state ($Z == 1$).
- If true → updates pc to label.

Complete Conditional Pattern

```
cmp x1, x2          // 1. Compare x1 and x2
b.eq equal_path     // 2. If x1 == x2, branch to equal_path
sub x0, x1, x2      // Fall-through: executed if x1 != x2
b end_block         // Bypass true-block
equal_path:
    add x0, x1, x2   // True-block: executed if x1 == x2
end_block:
```

Optimized Branching: cbz and cbnz

AArch64 introduces instructions that combine testing and branching into a single cycle:

Compare and Branch on Zero

`cbz x0, target_label`

- Tests if register x0 is equal to 0.
- If $x0 == 0 \rightarrow$ branch to label.
- **Does not alter NZCV flags.**

Compare and Branch on Non-Zero

`cbnz x0, target_label`

- Tests if register x0 is non-zero.
- If $x0 \neq 0 \rightarrow$ branch to label.
- Highly efficient for loop termination.

Microarchitectural Advantage

Fuses comparison and control-flow evaluation into a single instruction without clobbering condition flag state.

Summary & Self-Test

Topic 2 Comprehensive Summary

- **RISC Design Philosophy:** Emphasizes simple control circuitry, regular instruction formats, fixed-width encodings, and high energy efficiency.
- **Load/Store Architecture:** Strict isolation of memory access (`ldr`, `str`) from computation (`add`, `sub`).
- **AArch64 Register File:** 31 visible 64-bit registers (`x0–x30`), zero register `xzr`, frame pointer `x29`, and link register `x30` (`lr`).
- **Dedicated Program Counter (`pc`):** `pc` is not a general-purpose register and cannot be accessed as an arithmetic operand.
- **PSTATE and Flags:** The `cmp` instruction updates **N**, **Z**, **C**, **V** flags to drive conditional branches (`b.eq`, `b.ne`, `b.lt`).
- **Instruction Classes:** All AArch64 instructions partition into Data Processing, Memory Access, or Control Flow.

Self-Test: Questions 1 & 2

Question 1: RISC vs. CISC Principles

What are the main architectural principles that distinguish RISC from CISC? Why does simpler instruction decoding reduce processor power consumption?

Question 2: The Load/Store Sequence

How does the AArch64 instruction sequence below implement an update to a 64-bit memory variable? Why can't ARM perform this operation in a single arithmetic instruction?

```
ldr x1, [x0]    →    add x1, x1, #5    →    str x1, [x0]
```

Self-Test: Solutions 1 & 2

Solution 1

RISC uses fixed 32-bit instructions, a load/store model, and regular operand fields. CISC uses variable-length encodings and direct memory arithmetic. Simpler decoding reduces silicon gate count and dynamic switching activity ($P \propto C \cdot V^2 \cdot f$), lowering power draw.

Solution 2

The sequence explicitly stages memory mutation: `ldr` loads 64 bits into `x1`, `add` computes in the ALU, and `str` writes back to RAM. ARM cannot combine these because the ALU datapath is isolated from memory buses, adhering to the pure RISC load/store principle.

Self-Test: Questions 3 & 4

Question 3: AArch64 Registers and Special Roles

How many general-purpose registers does AArch64 expose? What are the functions of `xzr`, `sp`, `x30` (`lr`), and how is `pc` protected?

Question 4: Instruction Classes

What are the three core instruction classes in AArch64? Give one specific instruction example for each class.

Self-Test: Solutions 3 & 4

Solution 3

- x0–x30: 31 general-purpose 64-bit registers.
- xzr: Zero register (reads zero, writes discarded); sp: Dedicated Stack Pointer.
- x30 (lr): Link Register holding return addresses from bl.
- pc is **not** a general register and can only be updated via control-flow branches.

Solution 4

- ① **Data Processing:** add x3, x1, x2 (computes within registers).
- ② **Memory Access:** ldr x1, [x2] (loads 64-bit word from memory).
- ③ **Control Flow:** b.eq target or cbz x0, target (branching).

Self-Test: Question 5 & Solution

Question 5: Compare-and-Branch Execution

How do `cmp` and a conditional branch such as `b.lt` work together to implement high-level decision making in AArch64? What status field stores the intermediate outcome?

Self-Test: Question 5 & Solution

Question 5: Compare-and-Branch Execution

How do `cmp` and a conditional branch such as `b.lt` work together to implement high-level decision making in AArch64? What status field stores the intermediate outcome?

Solution 5

- `cmp x0, x1` performs internal subtraction ($x0 - x1$), discards the result, and updates the **N**, **Z**, **C**, **V** condition flags in **PSTATE**.
- `b.lt label` subsequently inspects the flags ($N \neq V$). If true, it loads the label address into `pc`; otherwise, it falls through to the next sequential instruction.
- Intermediate comparison outcome is stored in **PSTATE.NZCV**.